

PROJECT DOCUMENTATION

On

FIRE WEATHER INDEX PREDICTOR

A Machine Learning Model to Predict Fire Weather Index



Submitted by

Pavani Cheviti

Mentored by

Praveen

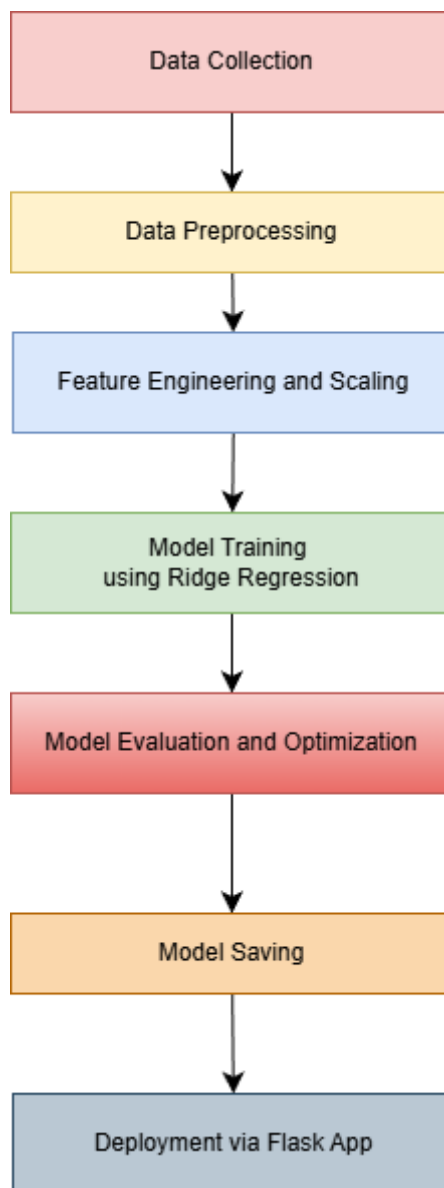
Project Statement:

Wildfires pose a significant threat to ecosystems, human life, and property. The Fire Weather Index (FWI) is a crucial tool used by meteorological and environmental agencies worldwide to estimate wildfire potential. This project aims to build a machine learning model that predicts FWI based on real-time environmental data, enabling proactive wildfire risk management. The model is trained using Ridge Regression, deployed via a Flask web application, and supports early warning systems for wildfire hazards.

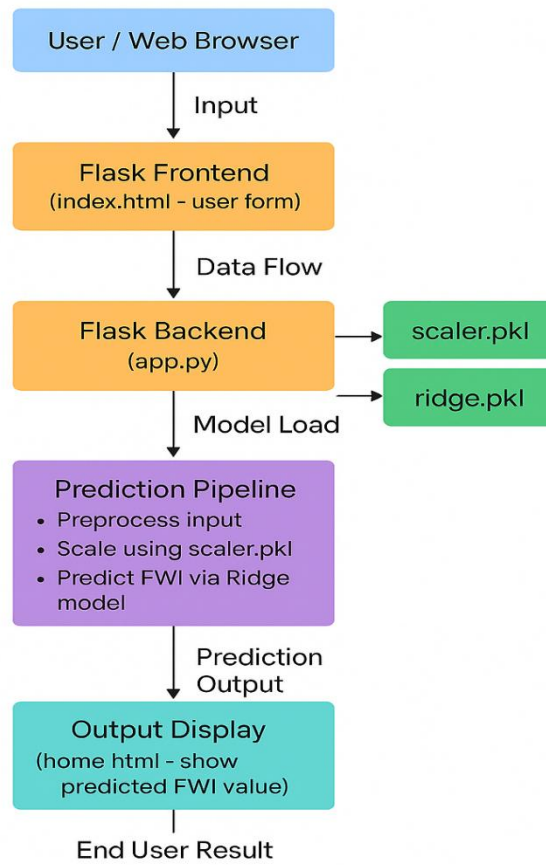
Outcomes:

- A predictive ML model trained using Ridge Regression to forecast FWI.
- A pre-processing pipeline using StandardScaler for normalization.
- A Flask-based web app where users can input environmental values and get FWI predictions.
- A system that can help forest departments, emergency planners, and climate researchers make data driven decisions.

Workflow of the FWI Prediction process



System Architecture of FWI Predictor



Milestone 1

Module 1: Data Acquisition and Cleaning

This module focused on getting the raw data ready for analysis. The dataset contained daily weather readings and various fire-related indices.

Environment Setup

- I have created a virtual environment inside VS Code.
- Installed required Python libraries: pandas, numpy, scikit-learn, matplotlib, flask, joblib

Data Collection, Loading and Initial Inspection

- I have downloaded the **FWI Dataset.csv** file. My dataset consists of the following attributes:
 - **Temperature** – Ambient temperature in °C
 - **Relative Humidity** – Percentage of moisture in the air
 - **Wind Speed** – Wind intensity in km/h
 - **Rain** – Precipitation in mm
 - **FFMC** – Fine Fuel Moisture Code
 - **DMC** – Duff Moisture Code
 - **ISI** – Initial Spread Index
 - **Region** – Categorical feature representing the region where data was recorded.
- I have loaded dataset into Pandas, ready for exploration
- I first looked at the first 10 rows and checked the structure **df.shape, df.info()**.
- I saw that almost all columns were incorrectly recognized as the object (text) data type and contained some missing values (NaNs).

Fixing Data Structure

- The dataset had some messy formatting.
 - I removed extra spaces from column names (like ' RH ' became 'RH').
 - I identified and deleted separator rows (rows 123-126) that were mistakenly introduced during data collection, which originally divided the two regions.

- I reset the row index to make it clean and sequential.
- I created a new column called region to clearly distinguish data from 'Bejaia' (first 122 days) and 'Sidi-Bel Abbes' (the rest).

Handling Missing Values and Duplicates

- I used **df.isnull().sum()** to find missing values.
 - I dropped all rows containing any missing values (**df.dropna()**). This simplified the dataset by ensuring all remaining entries were complete.
 - I removed any duplicate rows to ensure the data was unique and didn't bias my model.

Converting Data Types

- I converted the columns containing date information (day, month, year) and all the weather and fire-related metrics (Temperature, RH, Ws, Rain, FFMFC, DMC, DC, ISI, BUI, FWI) into proper numerical formats (int or float) so the machine learning algorithms could use them.
- I then decided that my goal was to predict the continuous FWI (Fire Weather Index) value, so the original Classes (fire/not fire) column was dropped.

Module 2: Data Exploration and Data Preprocessing

This module was about understanding the characteristics of the cleaned data, looking for patterns, and identifying potential problems.

Outlier Detection

- I have generated boxplots for all numerical features.
- Boxplots visually show the distribution of each feature and highlight outliers .
- I specifically calculated the count of outliers using the standard IQR (Interquartile Range) method. I found that

Outlier Count	
Day	0
month	0
year	0
Temperature	2
RH	0
Ws	8
Rain	34

FFMC	18
DMC	12
DC	14
ISI	4
BUI	11
FWI	4

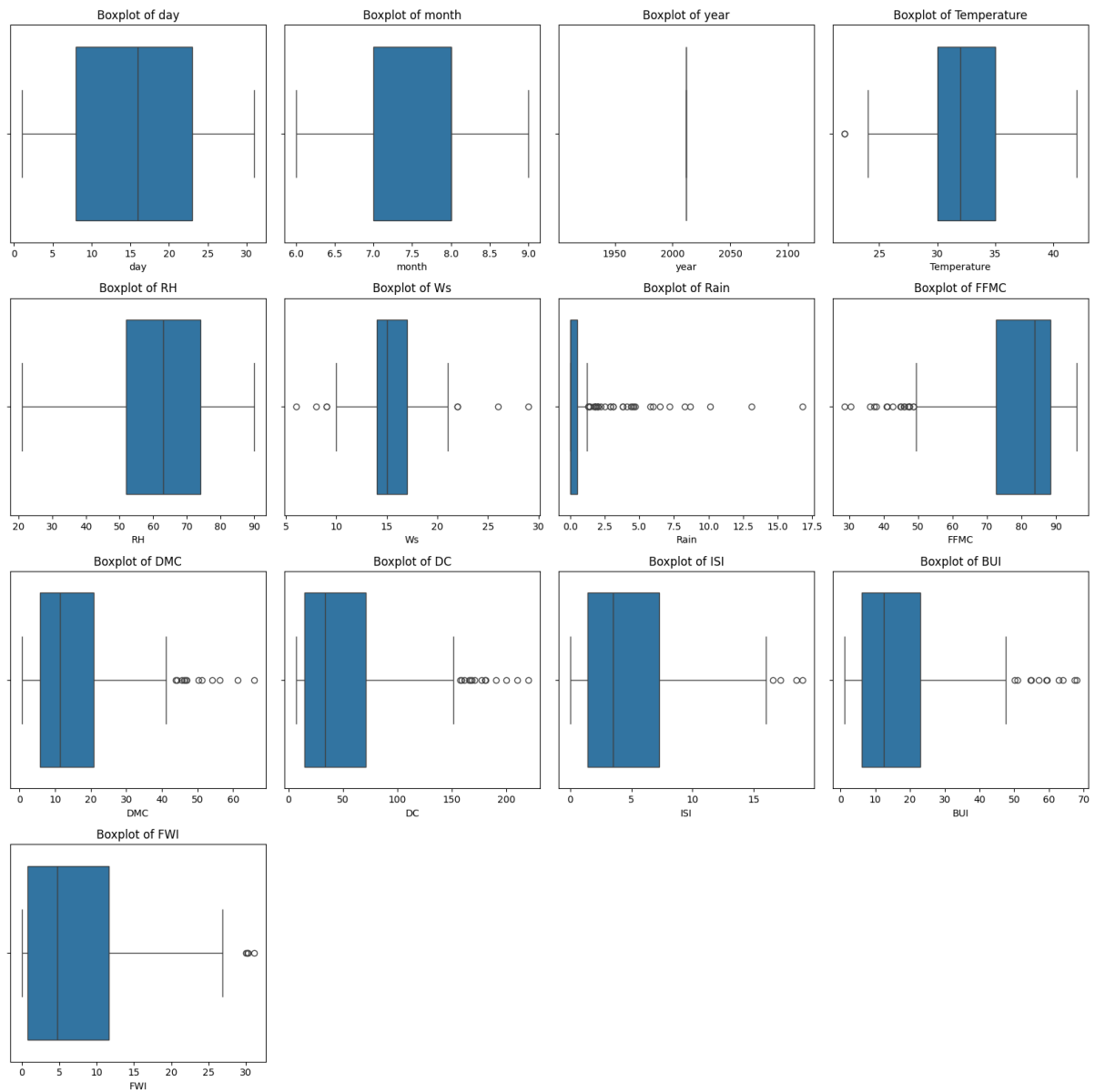


Figure 1: Boxplots of All Numerical Features. These plots show the spread and central tendency for each weather and fire-related variable. The dots outside the main boxes(whiskers) represent the outliers,

Checking Data Distribution

- I created **histograms** with a Kernel Density Estimate (KDE) for all numerical features.
- Histograms show how frequently different values occur. This helps confirm if the data is normally distributed (bell-shaped) or skewed.

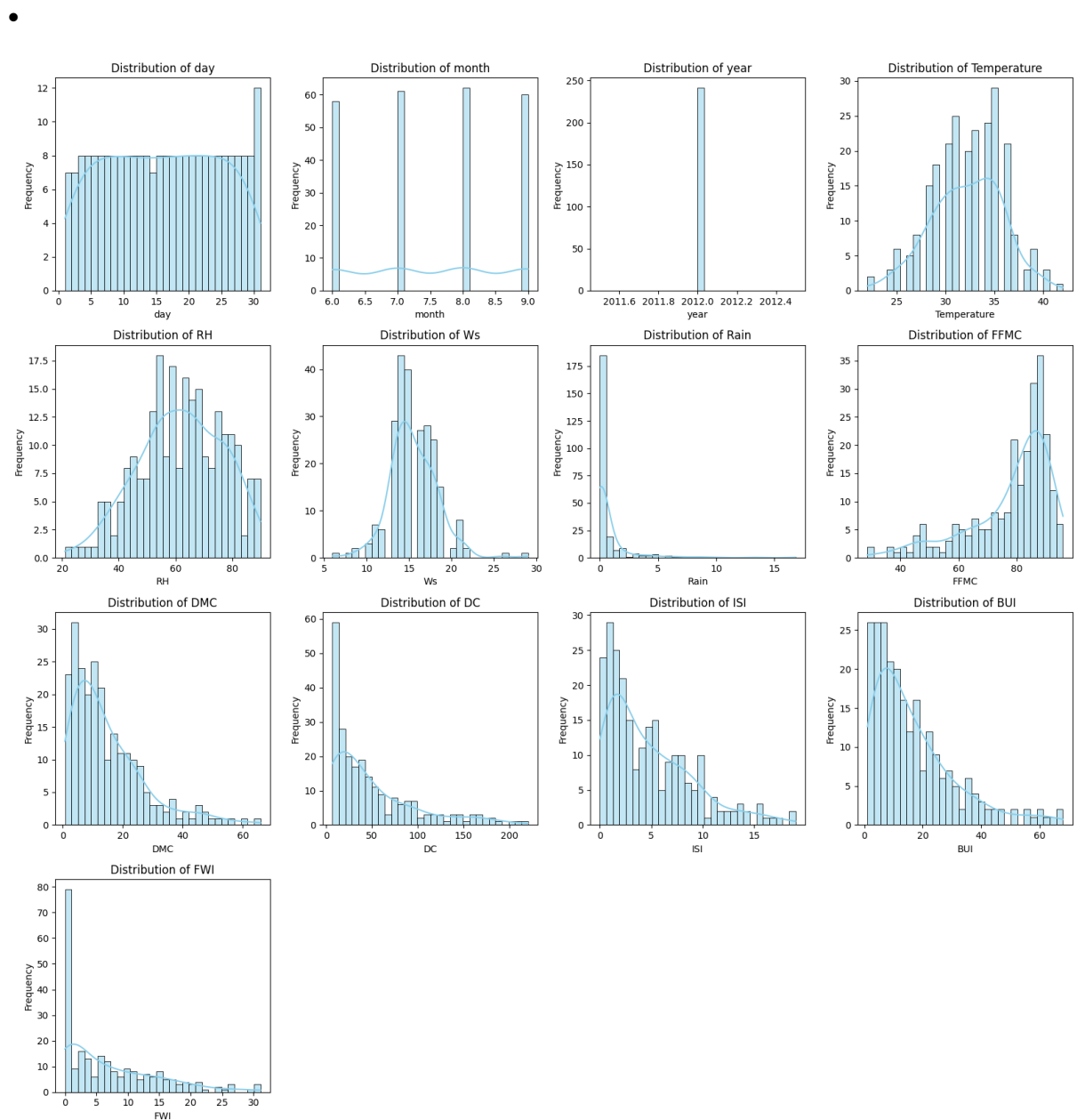


Figure 2: Data Distribution of All Features. Each plot displays a histogram and an overlaid Kernel Density Estimate (KDE). This visualization confirms how the values are spread.

Correlation Analysis

- I calculated the correlation between all numerical features and visualized it using a heatmap.
- This step is crucial for spotting multicollinearity (where two predictor variables are highly related) and identifying which weather factors are most strongly correlated with my target FWI.

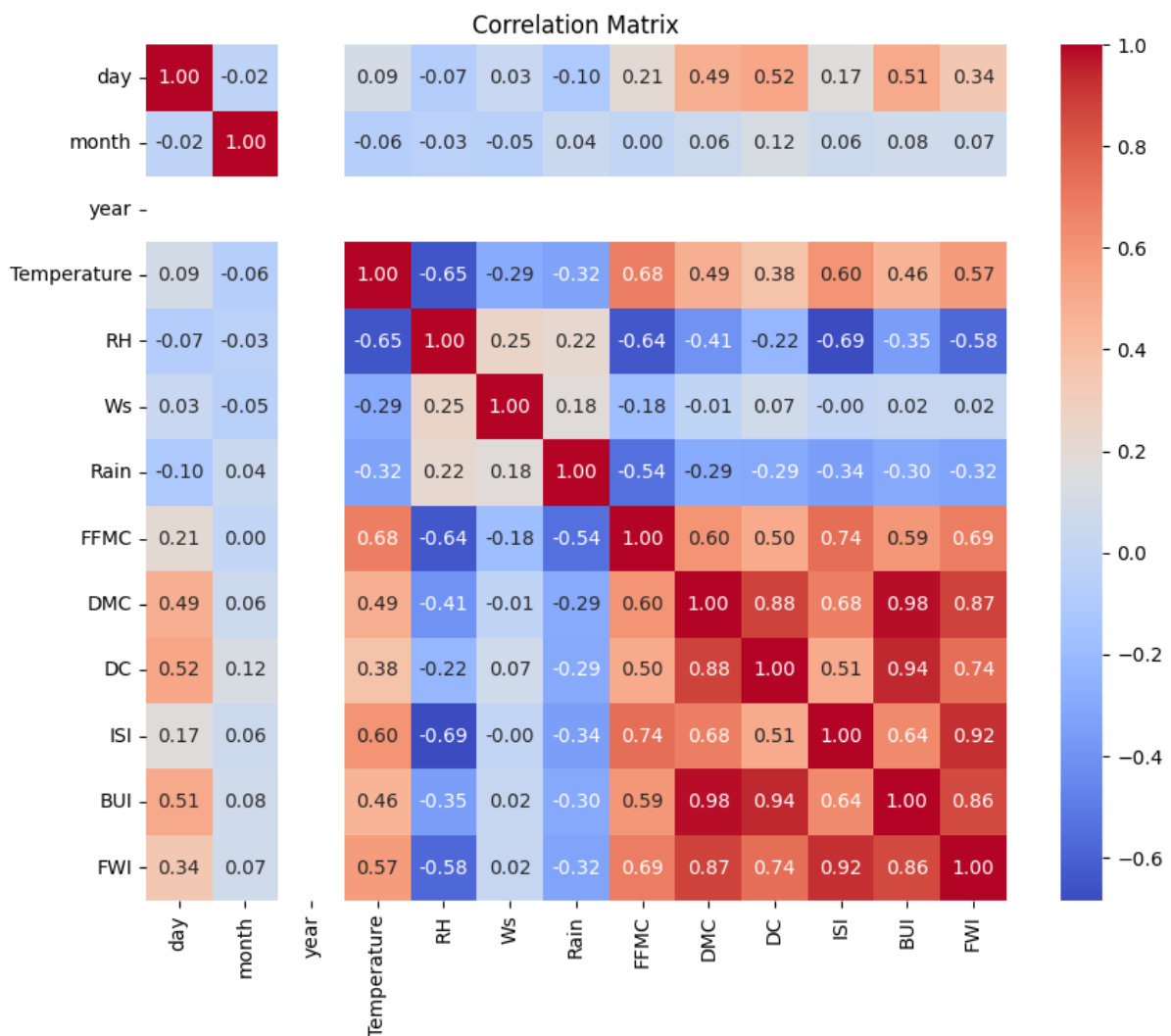
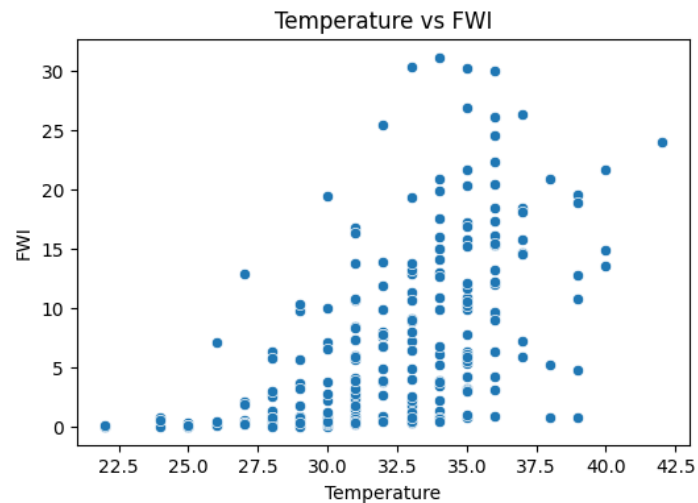


Figure 3: Correlation Matrix Heatmap. This map shows how strongly every feature relates to every other feature. The colors indicate the strength and direction of the relationship: dark colors (close to 1.0) mean a very strong positive link, and light colors (close to 0) mean a weak link.

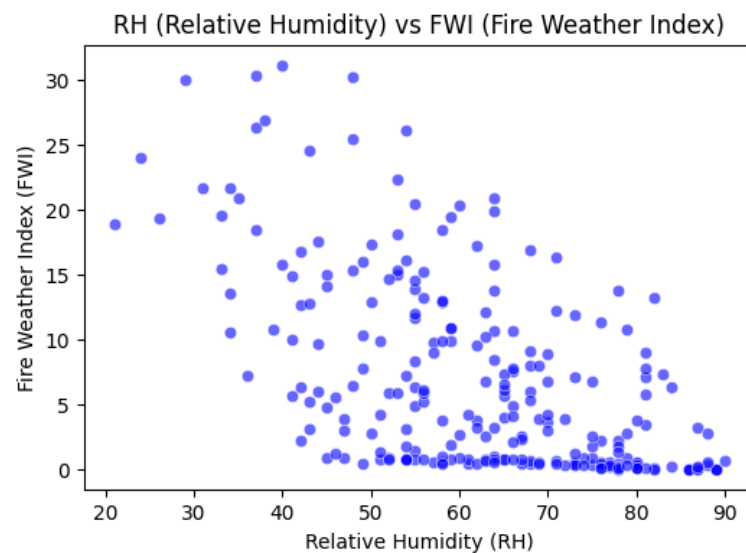
- **Visualizing Relationships with FWI (Scatterplots):**

- **Temperature vs FWI:** A scatterplot showed a positive trend: as temperature increases, FWI tends to increase.

-



- **RH vs FWI:** A scatterplot showed a strong negative trend: as relative humidity increases (meaning more moisture in the air), FWI tends to decrease.



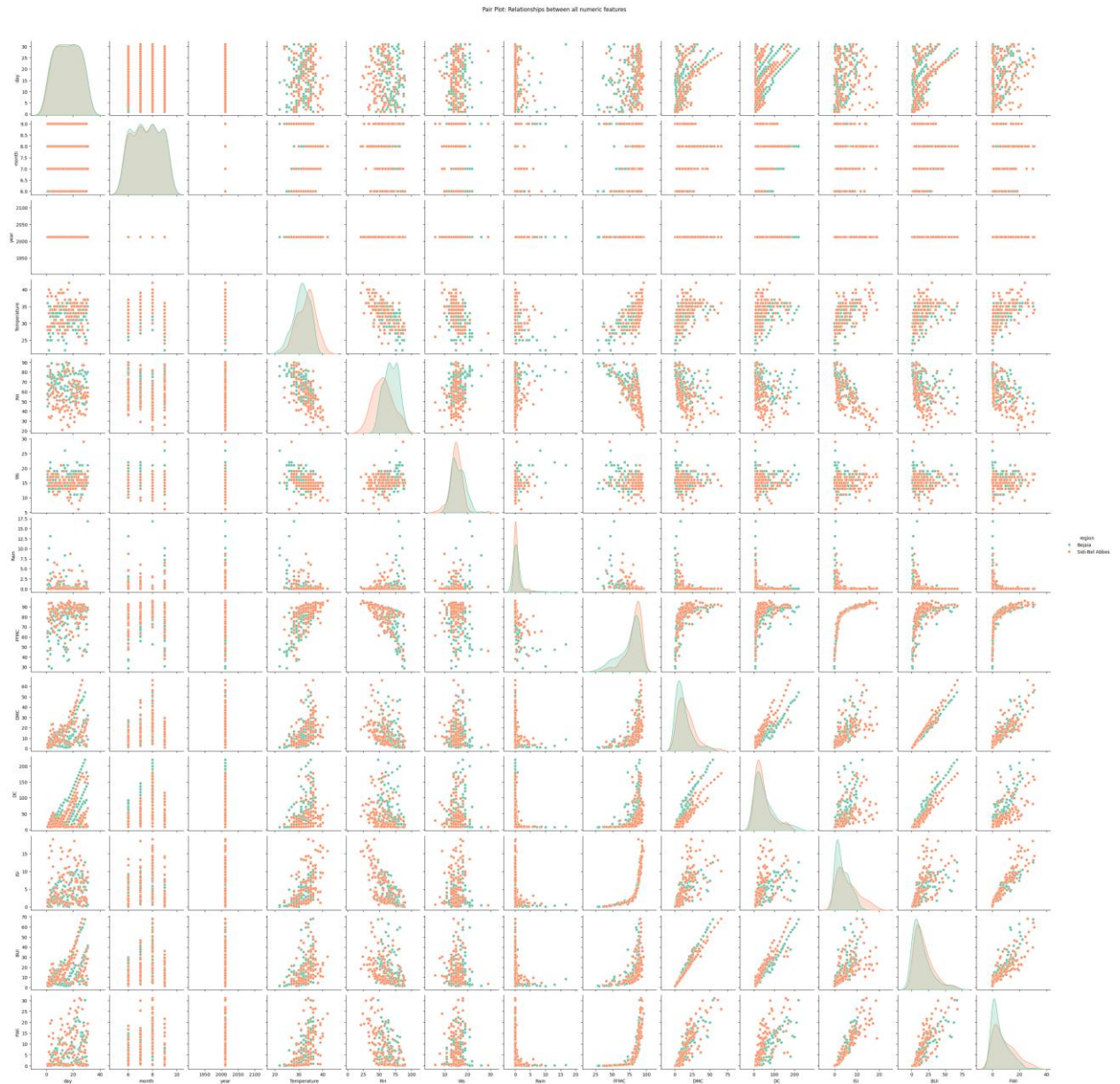


Figure 4: Pairwise Relationship Plots. This extensive grid of plots allows to visually check the relationship between every pair of numerical features simultaneously and observe the individual distribution of each feature.

Encoding Categorical Features

- Since the machine learning model can't directly understand the text in the region column (Bejaia/Sidi-Bel Abbès), I converted it into numerical form using label encoding. Encoding (creating two new columns, 'region_Bejaia' and 'region_Sidi-Bel Abbès', with 0s and 1s).

Milestone 2

Module 3: Feature Engineering and Model Preparation

I need to prepare the features to be perfectly suitable for the final model training process.

Features Selection

- Based on the high correlations observed in Module 2, I selected the key features most correlated with the FWI target variable. This step streamlines the model by focusing only on the most influential input features.

Selected features based on correlation: ['Temperature', 'RH', 'Ws', 'Rain', 'FFMC', 'DMC', 'DC', 'ISI', 'BUI', 'FWI']

Feature Scaling (Standardization)

- I normalized the numerical features using the **StandardScaler** tool from the *sklearn* library.
- This is a critical step that ensures all feature values have a consistent scale (zero mean and unit variance). It prevents features with naturally large values (like DC or BUI) from dominating the calculation of feature importance and ensures my models learn fairly from all inputs.

Splitting Data

- I separated the scaled data into training and testing sets using the ***train_test_split*** function.
 - **Training Set (80%):** Used to teach the model.
 - **Testing Set (20%):** Used to evaluate how well the trained model performs on new, unseen data.

Scaler Saving

- To maintain preprocessing consistency during model deployment, the fitted **StandardScaler** object was saved as a **.pkl** file using the *joblib* library. This ensures that the same scaling transformation is applied to new input data during prediction, providing uniformity between the training and deployment environments

Module 4: Model Training using Ridge Regression

This module focused on training and evaluating the Ridge Regression model, which is a variation of Linear Regression designed to perform well on datasets that exhibit multicollinearity (highly correlated features), like mine.

Model Selection and Training

- I chose the Ridge Regression model to address the high correlations (multicollinearity) I observed between my input features .
- Ridge Regression works by adding a penalty (L2 regularization) to the size of the model coefficients. This slight penalty helps prevent the model from becoming too complex or overfitting the training data.
- I trained the Ridge Regression model using the prepared **X_train** and **y_train** datasets.

Hyperparameter Tuning

- I tuned the alpha parameter (the strength of the regularization penalty).
- Selecting the optimal alpha is essential for the bias-variance tradeoff.
 - A very large alpha increases bias (model is too simple, underfits).
 - A very small alpha reduces the penalty, increasing the risk of variance (model is too complex, overfits).
 - I iterated to find the alpha value that yielded the best performance on the testing data while maintaining simple model coefficients.

Best alpha: 10

Best Ridge model: Ridge(alpha=10)

Performance Evaluation and Saving

- I monitored and evaluated the performance of the model throughout the training and validation phase.
- The performance was primarily assessed using the R^2 Score (closer to 1.0 is better). I used the training set performance to check for learning and the validation (testing) set performance to check for generalization.

Training R^2 : 0.9716601627949089

Testing R^2 : 0.9877420400985277

- After training and optimizing, I saved the final trained Ridge Regression model using the pickle library as ***ridge.pkl***.

Model saved as ridge.pkl

Training and Comparison

- I also trained model from each major regression family basic regularized (Ridge, Lasso, ElasticNet), and ensemble/tree-based (Decision Tree, Random Forest, Gradient Boosting).
- The performance of each model was measured using the R^2 Score on the unseen test data.

Model Type	Specific Algorithm	R^2 Score	Summary of Performance
Regularized	Ridge Regression	0.9716	Used L2 regularization; performance matched the basic linear model.
Regularized	Lasso Regression	0.9657	Used L1 regularization; performed slightly lower than Ridge.
Regularized	ElasticNet	0.9686	Combined L1 and L2 regularization; performed similarly to Lasso.
Tree-Based	Decision Tree Regressor	0.9549	The lowest performer among the tested models, though still highly accurate.
Ensemble	Random Forest Regressor	0.9834	A powerful ensemble method, his ensemble model achieved the highest R^2 Score .
Ensemble	Gradient Boosting Regressor	0.9789	This model also performed well, close to results of the Random Forest Regressor.

Among these, **Random Forest Regressor** initially achieved the **highest R^2 score**, indicating a strong fit on the training data.

After finding training ,validation testing of R²and RMSE of Random forest ,I got as following

<i>Random Forest Performance</i>		
<i>Dataset</i>	<i>R² Score</i>	<i>RMSE</i>
<i>Training</i>	0.9941	0.5747
<i>Validation</i>	0.9784	1.2287
<i>Testing</i>	0.9828	0.7964

Interpretation

- The Random Forest model achieved a **very high R² on the training set (0.9941)** compared to the validation (0.9784) and testing (0.9828) sets.
- The significant difference between **training RMSE (0.57)** and **validation RMSE (1.23)** suggests that the model learned the training data patterns too well, resulting in **overfitting**.
- This overfitting indicates poor generalization to unseen data, even though the testing R² remains high.

To address this issue and improve model generalization, **Ridge Regression** was chosen as the final model for evaluation and optimization. Ridge helps reduce overfitting by applying L2 regularization, penalizing large coefficient values.

Milestone 3

Module 5: Evaluation and Optimization

Evaluation Process

In this stage, the model was evaluated using multiple performance metrics and visualization techniques to assess its predictive accuracy and generalization.

1. **Evaluated the model using Mean Absolute Error (MAE):**
MAE measures the average magnitude of errors between predicted and actual values, showing how close the predictions are to the real data.

$$MAE = \frac{1}{n} \sum |y_{true} - y_{pred}|$$

2. **Computed Root Mean Squared Error (RMSE):**
RMSE gives higher weight to large errors, helping to penalize significant deviations.

$$RMSE = \sqrt{\frac{1}{n} \sum (y_{true} - y_{pred})^2}$$

3. **Calculated R² Score:**
The coefficient of determination (R²) explains how much variance in the target variable is captured by the model.

$$R^2 = 1 - \frac{SS_{res}}{SS_{tot}}$$

Performance Metrics of Ridge Regression

Dataset	R ² Score	RMSE	MAE
Training	0.9628	1.4474	0.9265
Validation	0.9792	1.2055	0.8827
Testing	0.9597	1.2175	0.8697

Interpretation:

- The R^2 values (≈ 0.96 – 0.98) show that the model explains over 95% of the variance in FWI.
- The low and consistent RMSE and MAE across all datasets indicate good generalization.
- The Ridge Regression model performs reliably without signs of overfitting.

Visualized Model Performance:

The purpose of this visualization is to compare the model's predicted Fire Weather Index (FWI) values against the actual observed FWI values. This helps evaluate how accurately the model performs in predicting unseen data.

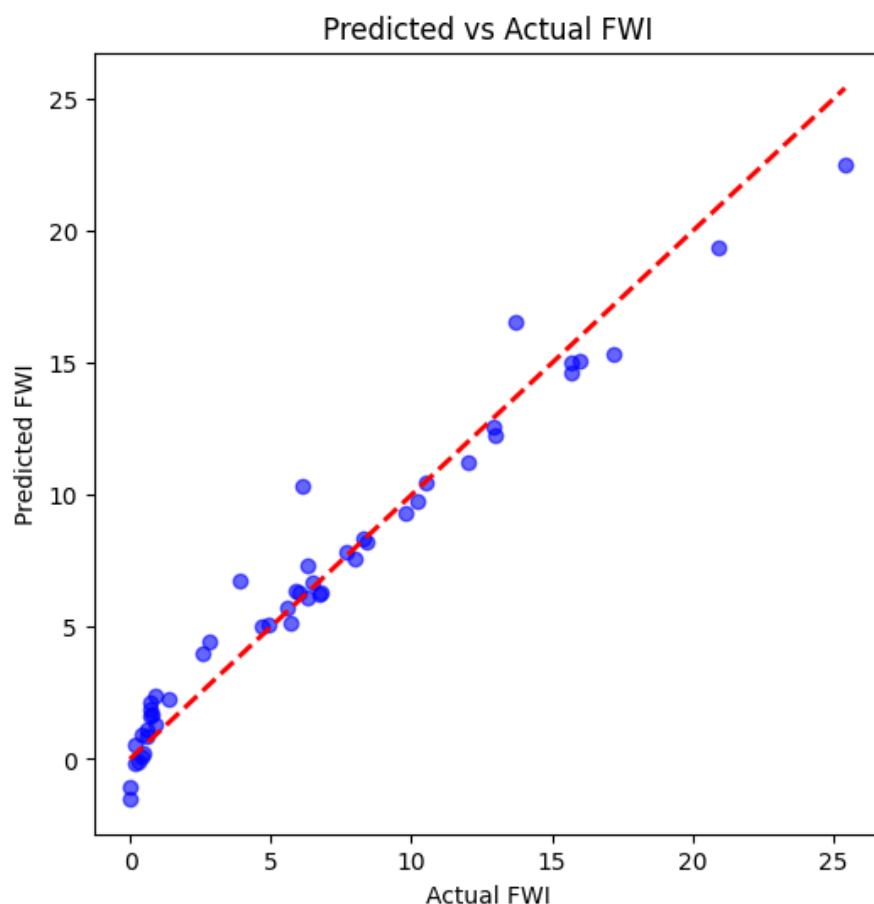


Figure 5.1: Predicted vs Actual Fire Weather Index (FWI) Plot

The scatter plot shows a strong positive correlation between predicted and actual values. The red dashed line represents the ideal 1:1 relationship, confirming the high accuracy of the Ridge Regression model.

`plt.scatter(y_test, y_test_pred)` :Plots the predicted vs. actual data points in blue, showing how predictions align with true values.

Interpretation

- Points that lie **close to the red dashed line** indicate accurate predictions.
- The **tight clustering around the diagonal** shows that the model's predictions closely match the actual FWI values.
- Only minor deviations are visible, suggesting **low prediction error** and **high generalization capability** of the model.
- This visualization confirms that the tuned Ridge Regression model performs effectively on the test dataset.

Hyperparameter Tuning and Model Optimization

To enhance the performance of the Ridge Regression model, the regularization parameter alpha (α) was tuned. This parameter controls the amount of L2 regularization applied to the model, helping to prevent overfitting and improve generalization.

- A range of alpha values was tested — [0.001, 0.01, 0.1, 0.5, 1, 10, 100].
- For each value, the model was trained and evaluated using the Root Mean Squared Error (RMSE) metric.
- The goal was to identify the alpha that produced the lowest RMSE, indicating the best predictive accuracy.

Output:

Alpha	RMSE
0.001	0.9672
0.01	0.9613
0.1	0.9305
0.5	0.9129
1	0.9219
10	1.2055
100	2.5396

Best Alpha: 0.5

Lowest RMSE: 0.9129

Interpretation

- As alpha increased from **0.001 to 0.5**, RMSE gradually **decreased**, showing improved model stability.
- Beyond alpha = 1, RMSE increased significantly, indicating **over-regularization**, where the model became too constrained and underfitted the data.
- The **optimal alpha = 0.5** achieved the **lowest RMSE (0.9129)**, representing the best balance between bias and variance.
- The model was retrained with this optimal alpha value to finalize the evaluation metrics for Module 5.

Then saved model successfully as ridge.pkl

Milestone 4:

Module 6: Deployment via Flask App

- Created a Flask app structure with app.py as the main application.
- Developed index.html for the user input form with all feature fields.
- Created home.html to display the predicted FWI value to the user.
- Loaded ridge.pkl and scaler.pkl during app runtime for predictions.
- Handled form input collection, preprocessing, prediction, and output display.

User Interface

The screenshot displays the Igniscope web application interface, which is designed for predicting the Fire Weather Index (FWI). The interface is divided into two main sections: a left sidebar with educational content and a main input form.

Header: The top of the page features a banner image of a forest fire with the title "Igniscope — Fire Weather Index Predictor".

Left Sidebar (Educational Content):

- What is the FWI?**: The Forest Fire Weather Index (FWI) combines weather and fuel conditions to estimate fire behavior.
- Temperature**: Higher temperatures increase evaporation and fire risk.
- Relative Humidity**: Lower humidity means drier fuels and increased fire potential.
- Wind Speed**: Strong winds can rapidly spread fires and make them harder to control.
- Rainfall**: Recent rain reduces immediate fire danger.
- FFMC**: Fine Fuel Moisture Code — ease of ignition indicator.
- DMC**: Duff Moisture Code — indicates fuel availability.
- DC**: Drought Code — shows long-term dryness.

Main Input Form:

- Temperature (°C)**: Input field with value 23.
- Relative Humidity (%)**: Input field with value 12.
- Wind Speed (km/h)**: Input field with value 34.
- 24-Hour Rainfall (mm)**: Input field with value 23.
- Fine Fuel Moisture Code (FFMC)**: Input field with value 12.
- Duff Moisture Code (DMC)**: Input field with value 23.
- Drought Code (DC)**: Input field with value 12.
- Initial Spread Index (ISI)**: Input field with value 3.
- Buildup Index (BUI)**: Input field with value 2.

Action Button: A large orange button labeled "Calculate Fire Risk" is positioned at the bottom of the input form.

Sample Output

